

Combat Sports Marketplace

Full Stack Project Plan — Django 6 + React 18 + PostgreSQL 18

10 Phases | 48 Steps | 40+ API Endpoints | 30+ Frontend Pages | 50+ Components

1. Project Overview

A vertical marketplace platform connecting fighters, coaches, gyms, and vendors. Users can buy gear, book coaches, discover gyms, access premium training content, attend VVIP events, and communicate via E2EE private chat.

Tech Stack

Layer	Technology
Backend	Django 6 + Django REST Framework
Frontend	React 18 + Vite + Tailwind CSS
Database	PostgreSQL 18
Real-time Chat	Django Channels + WebSockets + Web Crypto API (E2EE)
Payments	Stripe (test mode)
Auth	JWT (SimpleJWT) + Google OAuth
Media	Cloudinary (or local file storage)
Maps	Leaflet + OpenStreetMap
Async Tasks	Celery + Redis

Who It's For

Persona	Need
Athlete / Fighter	Buy gear, book coaches, find gyms, train with programs, attend events
Coach	List services, manage bookings, earn 100% of session fees, build reputation
Gym Owner	List gym, sell day passes/memberships, host events
Vendor / Brand	Sell products, sponsor fighters, run limited drops
Admin	Manage platform, moderate content, view analytics

2. Project Structure

```
capstone/
├── backend/
│   ├── config/
│   │   ├── settings.py
│   │   ├── urls.py
│   │   ├── asgi.py
│   │   └── wsgi.py
│   ├── accounts/           # User, Profile, auth views
│   ├── products/          # Products, Cart, Orders
│   ├── coaches/           # CoachProfile, Booking
│   ├── gyms/              # Gym, Memberships, DayPass
│   ├── exercises/         # Exercise, TrainingProgram
│   ├── chat/              # E2EE private chat
│   ├── comments/          # Polymorphic comments
│   ├── reviews/           # Star ratings
│   ├── events/            # Events, Tickets, QR codes
│   ├── subscriptions/     # Tiered subs, discounts
│   ├── common/            # Shared permissions, pagination
│   ├── manage.py
│   ├── requirements.txt
│   └── .env
├── frontend/
│   ├── src/
│   │   ├── providers/     # Auth, Subscription, Cart, Chat
│   │   ├── pages/         # 30+ route-level pages
│   │   ├── components/    # 50+ reusable components
│   │   ├── hooks/         # useAuth, useChat, useE2EE, etc.
│   │   ├── utils/         # API client, crypto, helpers
│   │   ├── layouts/       # Main, Auth, Dashboard
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   ├── routes.jsx
│   │   └── index.css
│   ├── index.html
│   ├── vite.config.js
│   └── package.json
└── README.md
```

3. Phase 1 — Foundation

2 days Auth system + Profile + Layout

3.1 Database Setup

```
PGPASSWORD=2962 psql -U levi -h localhost -c "CREATE DATABASE capstone;"
```

3.2 Backend Setup Commands

```
cd capstone && mkdir backend && cd backend
python3 -m venv venv && source venv/bin/activate
pip install django djangorestframework djangorestframework-simplejwt \
  django-cors-headers channels django-ckeditor django-filter \
  django-allauth google-auth sendgrid psycpg2-binary \
  python-decouple pillow qrcode stripe celery cloudinary
pip freeze > requirements.txt
django-admin startproject config .
python manage.py startapp accounts products coaches gyms exercises \
  chat comments reviews events subscriptions common
```

3.3 Environment File (.env)

```
DB_NAME=capstone
DB_USER=levi
DB_PASSWORD=2962
DB_HOST=localhost
DB_PORT=5432
SECRET_KEY=django-insecure-...
DEBUG=True
ACCESS_TOKEN_LIFETIME_MINUTES=30
REFRESH_TOKEN_LIFETIME_DAYS=1
GOOGLE_CLIENT_ID=your-google-client-id
STRIPE_PUBLISHABLE_KEY=your-publishable-key
STRIPE_SECRET_KEY=your-secret-key
SENDGRID_API_KEY=your-sendgrid-key
FROM_EMAIL=noreply@combathub.com
FRONTEND_URL=http://localhost:5173
```

3.4 Django Settings (config/settings.py)

Setting	Value
AUTH_USER_MODEL	accounts.User
REST_FRAMEWORK	JWT auth, pagination, filter backends

SIMPLE_JWT	30min access, 1 day refresh, rotate
CORS_ALLOWED_ORIGINS	http://localhost:5173
CHANNEL_LAYERS	InMemoryChannelLayer (dev)
DATABASES	PostgreSQL from .env
EMAIL	SendGrid SMTP

3.5 User Model Fields (accounts/models.py)

Field	Type	Notes
email	EmailField	unique, USERNAME_FIELD
username	CharField	unique, nullable
password	CharField	hashed via set_password()
role	CharField	athlete/coach/gym_owner/vendor/admin
google_id	CharField	nullable
display_name	CharField	max 100
is_active	BooleanField	default True
is_staff	BooleanField	default False

3.6 Additional Models (Phase 1)

- **Profile** (OneToOne User): bio, avatar, phone, weight_class, height_ft/in, reach_in, stance
- **UsernameChange**: tracks changes for 2/month limit
- **SiteContent**: admin-editable pages (About Us, etc.)

3.7 Auth API Endpoints

```

POST /api/auth/register/
POST /api/auth/login/
POST /api/auth/google/
POST /api/auth/set-username/
POST /api/auth/password-reset/
POST /api/auth/password-reset/confirm/
POST /api/auth/refresh/
GET/PATCH /api/auth/me/
GET /api/auth/site-content/<key>/

```

3.8 Frontend Setup Commands

```
cd capstone
npm create vite@latest frontend -- --template react
cd frontend
npm install react-router-dom axios tailwindcss @tailwindcss/vite \
  @react-oauth/google react-helmet-async \
  @stripe/react-stripe-js @stripe/stripe-js \
  react-leaflet leaflet react-player recharts react-big-calendar
mkdir -p src/{providers,pages,components/ui,layouts,hooks,utils}
```

3.9 Frontend Phase 1 Pages

Page	Route	Description
Home	/	Hero section + feature cards
Login	/login	Email/username + password + Google Sign-In
Register	/register	Email + username + password + role selector
ForgotPassword	/forgot-password	Email input → reset link sent
ResetPassword	/reset-password/:token	Token from URL + new password
UsernameSetup	/username-setup	After Google signup, choose username
ProfileEdit	/profile	Avatar upload, bio, stats, username change
About	/about	Fetches admin-editable content

3.10 Provider

- **AuthProvider:** Context with user state, login(), register(), googleLogin(), logout(). On mount: checks localStorage for JWT, fetches user. Auto-refreshes token on 401.

3.11 Layouts

- **MainLayout:** Sticky dark navbar + footer + Outlet
- **AuthLayout:** Gradient dark background + centered white card

3.12 Deployment Phase 1

```
# Backend
cd backend && source venv/bin/activate
python manage.py makemigrations && python manage.py migrate
python manage.py createsuperuser && python manage.py runserver

# Frontend
cd frontend && npm run dev
```

4. Phase 2 — Marketplace

3 days Products, Cart, Orders, Stripe Checkout

4.1 Database Models

- **Category:** name, sport_tag, parent (self-FK)
- **Product:** name, description, brand, category, sport_tags, images, price, stock, limited_edition, serial_number, drop_date
- **ProductVariant:** product, size, color, material, weight_oz, stock, price_override
- **Drop:** product, launch_time, max_quantity, countdown_active
- **Cart:** user, items (JSON)
- **CartItem:** cart, product, variant, quantity
- **Order:** user, items, subtotal, total, status, shipping_address, payment_intent

4.2 API Endpoints

GET /api/products/<filters>	GET /api/drops/active/
GET /api/products/<id>	POST /api/drops/<id>/notify/
GET /api/categories/	GET /api/cart/
POST /api/products/<id>/favorite	POST /api/cart/add/
	DELETE /api/cart/items/<id>
POST /api/checkout/	PATCH /api/cart/items/<id>
POST /api/stripe/webhook/	
GET /api/orders/	GET /api/orders/<id>/

4.3 Frontend Pages

- **Shop.jsx** — Product grid + sidebar filters (sport, category, price range, brand, limited drops)
- **ProductDetail.jsx** — Image gallery, variant picker, price, stock, add-to-cart, comments, reviews
- **Cart.jsx** — Item list, quantity controls, totals, checkout button
- **Checkout.jsx** — Shipping address + Stripe Elements
- **OrderHistory.jsx** — Past orders list
- **OrderDetail.jsx** — Single order details

4.4 Key Components

ProductCard, ProductGrid, VariantPicker, DropTimer, CartProvider

5. Phase 3 — Coaches & Gyms

2 days Coach booking + Gym directory

5.1 Database Models

- **CoachProfile** (OneToOne User): bio, certifications, specialties, fighting_style, hourly_rate, location, lat/lng, online_only, years_experience
- **CoachCertification**: name, issuing_body, verified
- **CoachAvailability**: day_of_week, start_time, end_time
- **Booking**: user, coach, date, time, duration, session_type, status, price
- **Gym**: name, address, lat/lng, description, amenities, photos, class_schedule
- **GymMembership**: name, price, duration_days, perks
- **GymDayPass**: price, valid_for_hours

5.2 API Endpoints

GET /api/coaches/<filters>	GET /api/coaches/<id>/
GET /api/coaches/<id>/availability	POST /api/coaches/<id>/book/
GET /api/bookings/	PATCH /api/bookings/<id>/cancel/
GET /api/gyms/<filters>	GET /api/gyms/<id>/
GET /api/gyms/<id>/day-passes/	GET /api/gyms/<id>/memberships/
GET /api/gyms/nearby/	

5.3 Frontend Pages

- **CoachList** — Cards + search + filters
- **CoachDetail** — Bio, certifications, availability calendar, booking modal
- **GymList** — Cards + Leaflet map toggle
- **GymDetail** — Photos, amenities, day-pass/membership cards

6. Phase 4 — Subscriptions

2 days Tiered access + Stripe billing

6.1 Tier Model

Feature	Free	Premium \$19.99	VIP \$49.99	Coach \$29.99	Gym \$49.99
Browse marketplace					
Buy products					
Send DMs	3/ day	Unlimited	Unlimited	Unlimited	Unlimited
Exercise library	Text	Full HD	Full HD	Full HD	Full HD
Early drops access		24h	48h	—	—
Free shipping		\$50+	All	—	—
10% off shop				—	—
Keep 100% earnings	—	—	—		—
Profile badge	—	—	Gold+QR	Verified	Verified

6.2 API Endpoints

```
GET /api/subscriptions/plans/  
POST /api/subscriptions/subscribe/  
POST /api/subscriptions/cancel/  
POST /api/subscriptions/change/  
GET /api/subscriptions/my/  
GET /api/discounts/active/
```

6.3 Frontend

- **Plans.jsx** — Tier comparison table with trial CTA
- **ManageSub.jsx** — Current tier, cancel/change, discount code
- **SubscriptionGate** — Wrapper component for premium content
- **TrialBanner** — Days remaining in trial
- **DiscountBanner** — Seasonal deals

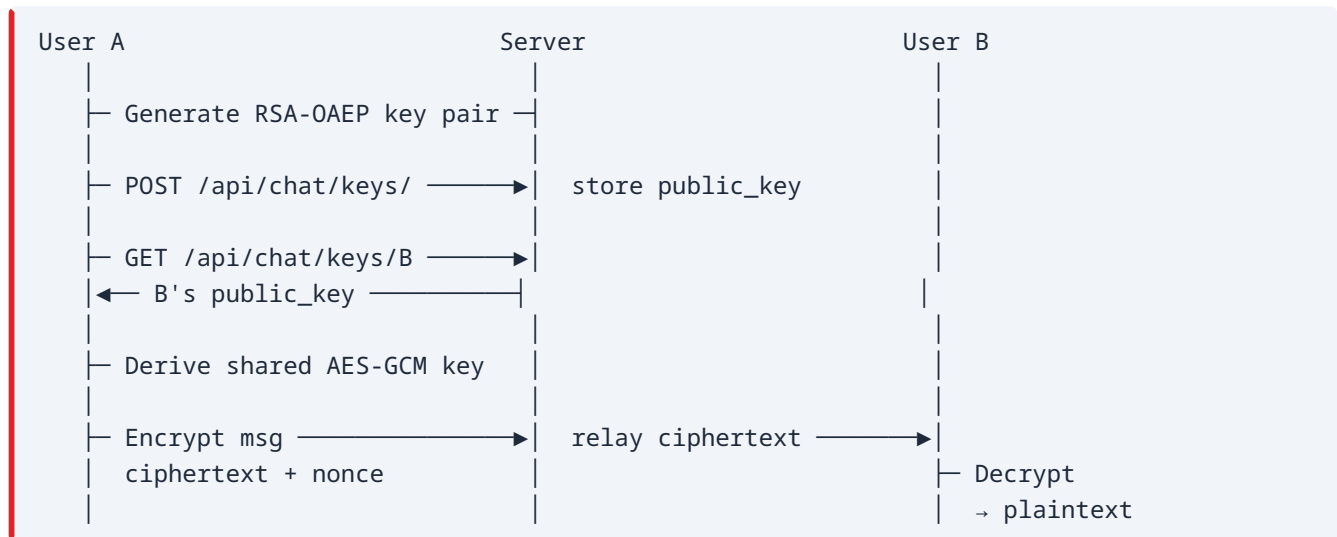
6.4 Stripe Integration

- Subscribe: `stripe.Subscription.create(trial_period_days=30)`
- Cancel: at period end
- Change: upgrade/downgrade via Stripe API
- Webhooks: `invoice.paid`, `customer.subscription.*` → sync to DB
- Discounts: Stripe Coupon API

7. Phase 5 — E2EE Chat

2 days Private encrypted messaging

7.1 Architecture



7.2 Models

- **ChatConversation:** participant1, participant2, created_at, last_message_at
- **ChatMessage:** conversation, sender, encrypted_body, nonce, created_at, read
- **UserKey:** user, public_key (JWK), created_at, rotated_at

7.3 WebSocket

```
WS /ws/chat/<conversation_id>/?token=<jwt>
```

Consumer validates JWT on connect, adds user to conversation group.

7.4 Client-Side Crypto (Web Crypto API)

- **generateKeyPair()** → RSA-OAEP 2048-bit
- **encrypt(plaintext, key)** → { ciphertext, nonce }
- **decrypt(ciphertext, nonce, key)** → plaintext
- Private key stored in IndexedDB (never leaves browser)

7.5 Frontend

- **ChatList.jsx** — Conversation list with preview + unread badge
- **ChatRoom.jsx** — Message history, encrypted indicator, input field

- **useE2EE.js** — Key lifecycle, encrypt/decrypt
- **useChat.js** — WebSocket connect/disconnect, send/receive

8. Phase 6 — Training Library

2 days Exercise library + workout programs

8.1 Models

- **Exercise:** name, sport, description, muscle_group, equipment, difficulty, video_url, created_by
- **TrainingProgram:** name, description, sport, duration_weeks, goal, price_tier
- **Workout:** program, week_number, day_number, exercises (JSON: sets/reps/rest)
- **UserProgress:** user, workout, date, completed, notes

8.2 API

```
GET /api/exercises/<filters>      GET /api/exercises/<id>/
GET /api/training-programs/      GET /api/training-programs/<id>/
GET /api/training-programs/<id>/workouts/
POST /api/workouts/<id>/log/      GET /api/progress/
```

8.3 Frontend

- **ExerciseLib** — Filterable grid, locked content for free users
- **ExerciseDetail** — Video player, description, tags
- **Programs** — Program cards with progress bars
- **ProgramDetail** — Week/day view, log button
- **TrainingLog** — Calendar, streaks, charts

9. Phase 7 — Events & Ticketing

1 day VVIP/vip/public event ticketing with QR codes

9.1 Models

- **Event:** name, type (vvip/vip/public), date, location, max_attendees, cover_image, host
- **EventTicket:** event, tier, price, perks, quantity_available
- **TicketPurchase:** user, event_ticket, quantity, qr_code, status
- **EventBundle:** event + products + training_session, discount_pct

9.2 API

```
GET /api/events/                GET /api/events/<id>/
GET /api/events/<id>/tickets  POST /api/events/<id>/purchase/
GET /api/events/<id>/bundles  POST /api/events/<id>/bundles/<id>/purchase/
GET /api/tickets/my/
```

9.3 Frontend

- **EventList** — Upcoming events, filter by type
- **EventDetail** — Cover image, description, ticket tiers, bundles
- **TicketView** — QR code display, event countdown

10. Phase 8 — Comments & Reviews

1 day Social features (polymorphic comments + star ratings)

10.1 Models

- **Comment:** user, body, parent (threaded), GenericForeignKey (Product/Coach/Gym/Exercise)
- **Review:** user, body, rating 1-5, GenericForeignKey

10.2 API

```
GET /api/comments/?target_type=&target_id=  
POST /api/comments/ → { target_type, target_id, body, parent }  
DELETE /api/comments/<id>/  
GET /api/reviews/?target_type=&target_id=  
POST /api/reviews/ → { target_type, target_id, rating, body }
```

10.3 Components

- **CommentThread** — Recursive threaded replies
- **CommentForm** — Textarea + submit + target selector
- **StarRating** — 1-5 star selector, hover effect
- **ReviewForm** — Stars + text + image upload

11. Phase 9 — Dashboards & Extras

2 days User dashboards, admin panel, gamification, sponsorship

11.1 Extra Models

- **SweatPoints**: balance, lifetime_earned, transactions
- **GymCheckIn**: user, gym, date (Fighter Passport)
- **SponsorshipListing**: fighter profile, record, asking
- **SponsorshipDeal**: brand → fighter deal

11.2 API

```
GET /api/users/me/dashboard/
GET /api/users/me/sweat-points/      POST /api/users/me/sweat-points/redeem/
GET /api/users/me/favorites/
GET /api/users/me/passport/
POST /api/gyms/<id>/check-in/
GET /api/sponsorships/listings/      POST /api/sponsorships/listings/
```

11.3 Frontend Pages

- **Dashboard** — Stat cards, activity feed, favorites
- **SweatShop** — Points balance, redeem options
- **Favorites** — 3 tabs (Products/Coaches/Gyms)
- **Passport** — Check-in history with stamps + map
- **CoachDashboard** — Bookings, availability editor, analytics
- **GymDashboard** — Pass/membership sales, analytics
- **AdminPanel** — Users, products, orders, moderation queue

12. Phase 10 — Polish & Deploy

2 days Production readiness

12.1 Tasks

- Seed data script (sample users, products, coaches, gyms, exercises, events)
- Error boundaries on every page
- Toast notifications for API errors/success
- Skeleton loaders for all list pages
- Dark mode toggle (Tailwind class strategy)
- Responsive design (mobile nav, stacked grids)
- SEO meta tags via react-helmet-async

12.2 Deployment

Service	Backend	Frontend
Host	Render / Railway / fly.io	Vercel / Netlify
Runtime	Gunicorn + Daphne (ASGI)	Static build (npm run build)
Database	Managed PostgreSQL	—
Cache	Redis (Channels + Celery)	—
Env vars	Dashboard secrets	VITE_API_URL + VITE_GOOGLE_CLIENT_ID

13. Complete API Reference

Auth (accounts)

POST /api/auth/register/	POST /api/auth/login/
POST /api/auth/google/	POST /api/auth/set-username/
POST /api/auth/password-reset/	POST /api/auth/password-reset/confirm/
POST /api/auth/refresh/	GET/PATCH /api/auth/me/
GET /api/auth/site-content/<key>/	

Products

GET /api/products/<filters>	GET /api/products/<id>/
GET /api/categories/	GET /api/drops/active/
POST /api/drops/<id>/notify/	POST /api/products/<id>/favorite/

Cart & Orders

GET /api/cart/	POST /api/cart/add/
DELETE /api/cart/items/<id>/	PATCH /api/cart/items/<id>/
POST /api/checkout/	POST /api/stripe/webhook/
GET /api/orders/	GET /api/orders/<id>/

Coaches

GET /api/coaches/<filters>	GET /api/coaches/<id>/
GET /api/coaches/<id>/availability	POST /api/coaches/<id>/book/
GET /api/bookings/	PATCH /api/bookings/<id>/cancel/

Gyms

GET /api/gyms/<filters>	GET /api/gyms/<id>/
GET /api/gyms/<id>/day-passes/	GET /api/gyms/<id>/memberships/
GET /api/gyms/nearby/	POST /api/gyms/<id>/check-in/

Training

GET /api/exercises/<filters>	GET /api/exercises/<id>/
GET /api/training-programs/	GET /api/training-programs/<id>/
GET /api/training-programs/<id>/workouts/	
POST /api/workouts/<id>/log/	GET /api/progress/

Chat

GET /api/chat/conversations/	POST /api/chat/conversations/
GET /api/chat/conversations/<id>/messages/	
POST /api/chat/conversations/<id>/messages/	
GET /api/chat/keys/<user_id>/	POST /api/chat/keys/
WS /ws/chat/<conversation_id>/	

Comments & Reviews

GET /api/comments/<filters>	POST /api/comments/
DELETE /api/comments/<id>/	GET /api/reviews/<filters>
POST /api/reviews/	

Events

GET /api/events/	GET /api/events/<id>/
GET /api/events/<id>/tickets/	POST /api/events/<id>/purchase/
GET /api/events/<id>/bundles/	POST /api/events/<id>/bundles/<id>/purchase/
GET /api/tickets/my/	

Subscriptions

GET /api/subscriptions/plans/	POST /api/subscriptions/subscribe/
POST /api/subscriptions/cancel/	POST /api/subscriptions/change/
GET /api/subscriptions/my/	GET /api/discounts/active/

Dashboard & Extras

GET /api/users/me/dashboard/	GET /api/users/me/sweat-points/
POST /api/users/me/sweat-points/redeem/	
GET /api/users/me/favorites/	GET /api/users/me/passport/
GET /api/sponsorships/listings/	POST /api/sponsorships/listings/
GET /api/sponsorships/deals/my/	

14. Complete Frontend Routes

/	Home
/about	About
/login	Login
/register	Register
/forgot-password	ForgotPassword
/reset-password/:token	ResetPassword
/username-setup	UsernameSetup
/profile	ProfileEdit
/shop	Shop (product list)
/shop/:id	ProductDetail
/cart	Cart
/checkout	Checkout
/orders	OrderHistory
/orders/:id	OrderDetail
/coaches	CoachList
/coaches/:id	CoachDetail
/gyms	GymList
/gyms/:id	GymDetail
/training	ExerciseLibrary
/training/:id	ExerciseDetail
/training/programs	Programs
/training/programs/:id	ProgramDetail
/training/log	TrainingLog
/chat	ChatList
/chat/:id	ChatRoom
/events	EventList
/events/:id	EventDetail
/events/tickets/:id	TicketView
/subscriptions	Plans
/subscriptions/manage	ManageSub
/dashboard	Dashboard
/dashboard/favorites	Favorites
/dashboard/passport	Passport
/dashboard/sweat	SweatShop
/dashboard/bookings	Bookings
/coach/dashboard	CoachDashboard
/gym/dashboard	GymDashboard
/admin	AdminPanel

15. Build Timeline

Week	Phase	Deliverable
1	Phase 1	Auth system + profile + layout working
2-3	Phase 2	Product marketplace with cart & Stripe checkout
4-5	Phase 3	Coach booking + gym directory with Leaflet map
6-7	Phase 4	Subscription tiers + Stripe billing + trial flow
8-9	Phase 5	E2EE private chat with WebSockets
10	Phase 6	Exercise library + training programs + progress tracking
11	Phase 7-8	Events/ticketing (QR) + comments/reviews
12	Phase 9-10	Dashboards, admin panel, sweat points, polish, deploy

Design System (Tailwind)

Combat Sports Marketplace — Full Project Plan
Generated May 2026

Element	Value
Primary color	Red (#dc2626)
Primary hover	Dark red (#b91c1c)
Secondary (navbar/footer)	Dark navy (#1e293b)
Page background	Gray-50 (#f9fafb)
Cards	White, shadow-lg, hover:shadow-xl, hover:-translate-y-1
Auth pages	Gradient dark → centered white card
Buttons	rounded-lg, transition, hover effects
Inputs	Focus ring primary, rounded-lg